

김승환 | 개발 포트폴리오

슈프리마 Frontend Software Engineer 지원

010-9923-4036 / gloz9102@gmail.com

운영 제품의 기능 구현과 구조 개선

총 소프트웨어 개발 경력 **11년 이상**입니다. Java/Spring 기반 화면, API와 SQL 개발을 거쳐 2022.09부터 React와 Next.js 중심으로 의료인 대상 웹과 모바일 제품의 핵심 기능을 직접 구현하고 운영하고 있습니다.

2024.10부터 FE 파트리더 역할과 핵심 기능 구현을 병행했습니다. FE 팀원 3명과 공통 구현 기준을 조율했습니다. 리딩 범위는 기술 기준과 구현 검토이며, 인사 관리는 포함하지 않습니다.

경력과 이 문서의 구성

기간	담당 업무	상세
2019.07 - 현재	메디씨앤씨 / Java 운영에서 FE 개발로 확장	아래 병행 프로젝트
2024.10 - 현재	메디게이트 웹 전환, 공통 모듈	2쪽
2023.06 - 2026.07	개원입지 리포트, 웹과 PDF 출력	3쪽
2022.09 - 현재	메디게이트 모바일, 검색과 교육 영상	4쪽
2026.08	프레임워크와 API 전환 사전 검증	5-6쪽
2020.02 - 2023.05	Java/Spring 화면, API와 SQL	7쪽
2015.03 - 2019.03	젠타 / 웹 서비스와 백엔드 개발	7쪽

메디씨앤씨 입사 후 기존 Java/Spring 서비스 운영을 담당했습니다. 2022.09부터 React와 Next.js 중심 FE 개발을 담당했고, 2023.05까지 Java 개발을 병행했습니다. 위 프로젝트 기간은 서로 겹칩니다.

주요 기술

React 18, TypeScript, Next.js App Router, TanStack Query, Zustand, Redux Toolkit, React Hook Form, Zod, Turborepo, Puppeteer, pdf-lib, Java, Spring, JSP, MyBatis, SQL

적용 상태를 읽는 기준

운영 사례와 전환 사전 검증을 구분했습니다. 2026.08 전환 작업은 기록 당시 모바일 소스 편집만 통합 브랜치에 병합됐으며, 프레임워크 상향과 API 전환은 기능 브랜치 검증 단계입니다.

사례 1. 운영 중인 웹의 점진 전환

메디게이트 웹 / 2024.10 - 현재

역할: FE 파트리더로 커뮤니티, 채용과 임대분양의 화면 및 API 연동, 인증 클라이언트와 공통 모듈을 구현했습니다. 팀의 구현 기준을 조율했으며 서버 API와 서버 인증 정책은 백엔드 담당 범위입니다.

적용 상태: JSP와 Next.js가 병행하는 운영 서비스에 반영했습니다.

문제와 선택

기존 URL과 인증 흐름을 유지하면서 기능을 Next.js로 옮겨야 했습니다. UI와 타입, HTTP 기반 기능은 공통 패키지로 분리하고 앱별 인증 처리와 화면 정책은 각 앱에 남겼습니다.

제약	선택	관리할 비용
기존 서비스와 병행	기능 단위 전환	기존 URL과 인증 호환
앱마다 다른 인증 정책	공통 기반과 앱 정책 분리	공통 변경 시 앱별 영향 확인
여러 요청의 인증 만료	진행 중인 갱신 요청 공유	갱신 완료와 실패 분기

동시 인증 갱신 처리

같은 브라우저 실행 컨텍스트에서 여러 요청이 동시에 401을 받으면 진행 중인 인증 갱신을 공유하게 했습니다. 갱신 성공 후 원 요청의 인증 헤더를 교체해 재시도하고, 갱신이 실패하면 원 요청을 재시도 하지 않습니다.

여러 요청의 401

- 진행 중인 갱신 공유
- 성공: 헤더 교체 후 원 요청 재시도
- 실패: 원 요청 재시도 중단

상태의 수명에 따른 분리

포인트 화면의 탭은 URL로, 서버 응답은 TanStack Query로, 작성 중 입력은 React Hook Form과 Zod로 관리했습니다. 다시 진입할 때 유지할 상태와 서버에서 갱신할 데이터, 미완성 입력을 구분했습니다.

직접 URL 접근 시의 탭 노출 조건과 모바일 모달 이탈 흐름도 처리했습니다.

인증과 비인증 HTTP 클라이언트의 분리 범위를 팀과 논의했습니다. 2026.07 기준 코드는 분리를 유지합니다. 공통화 대상과 앱 정책의 경계를 실제 사용 조건에 맞춰 검토했습니다.

체크 오류 발생 시 새로고침하고 같은 탭의 반복 실행을 10초 간격으로 제한했습니다.

검증: 운영 코드에서 인증 갱신 공유와 실패 분기, 화면별 URL과 상태 연결을 확인할 수 있습니다.

결과: 기존 서비스와 병행하며 기능을 옮기는 구조와 인증 처리 흐름을 구현했습니다. 앱별 정책을 유지하면서 UI, 타입과 HTTP 기반 기능을 함께 사용할 수 있게 했습니다.

사례 2. 리포트의 화면과 PDF 출력

메디게이트 개원입지 / 2023.06 - 2026.07

역할: 리포트 화면과 표시 조건, PDF 생성 경로를 구현했습니다. 분석 데이터의 산출과 서버 API는 백엔드 담당 범위입니다.

적용 상태: 운영 서비스의 웹 리포트와 PDF 출력에 반영했습니다.

버전 호환과 로딩 방식

과거 리포트와 신규 리포트는 제공 필드와 표시 조건이 다릅니다. 공통 타입과 판별 함수로 버전을 구분하고, 복수 진료과와 버전별 표시 분기를 관리했습니다.

웹에서는 필요한 섹션을 지연 로딩하고, 전체 내용을 출력해야 하는 PDF는 정적 로딩을 유지했습니다. 데이터 버전의 표시 조건과 화면을 불러오는 방식을 분리해 관리했습니다.

리포트 버전 판별 → 버전별 표시 조건
└ 웹: 섹션별 지연 로딩
└ PDF: 정적 로딩, 페이지 출력

제약	선택	관리할 비용
과거와 신규 리포트 공존	타입과 판별 함수로 표시 분기	버전별 조건 유지
부분 조회와 전체 출력	웹 지연 로딩 / PDF 정적 로딩	출력 경로별 확인
반복 생성과 자원 보유	브라우저 재사용과 재시작 조건	페이지와 브라우저 수명 관리

생성 자원과 화면 상태 관리

Puppeteer 브라우저를 재사용하되 유휴 시간과 열린 페이지 수에 따라 재시작하도록 했습니다. 처리가 끝난 페이지를 닫고, 표지와 본문을 분리해 병합했으며 처리 단계별 시간을 기록했습니다.

지도 오버레이를 화면 수명에 맞춰 정리하고, Zustand selector로 구독 대상을 나눴습니다. 서버 데이터는 TanStack Query의 캐시 수명으로 관리했습니다.

날짜에 따른 가격 정책을 순수 함수로 분리하고 결제 실패 화면에서도 같은 정책을 사용했습니다. 화면마다 날짜 조건과 가격 계산을 반복하지 않도록 했습니다.

검증: 버전 판별과 표시 분기, 웹 지연 로딩과 PDF 정적 로딩, 페이지 종료와 브라우저 재시작 조건을 구현 기록에서 확인했습니다.

결과: 리포트 버전별 표시 조건을 유지하면서 웹 조회와 전체 PDF 출력의 요구를 각각 처리했습니다. 반복 생성에 필요한 자원 수명과 단계별 처리 시간을 확인할 수 있게 했습니다.

모바일 제품의 탐색과 교육 영상

메디게이트 모바일 / 2022.09 - 현재 / 운영 서비스

역할: 모바일 웹의 화면 구조 전환, 검색과 복귀 흐름, 교육 영상 화면을 구현했습니다. Flutter 앱 자체 개발과 영상 전송 서버는 이 사례의 담당 범위가 아닙니다.

Pages Router에서 App Router로 전환

2023.03 - 2023.05에 Pages Router 구조를 App Router로 전환했습니다. 기능별 app과 공용 shared 경계로 코드를 재배치하고 로그인과 홈 진입점을 옮겼습니다. Next.js 13과 React 18 기반의 운영 구현입니다.

검색 조건과 복귀 위치 복원

상세 화면이나 작성 화면으로 이동한 뒤 목록으로 돌아오면 검색 조건과 탐색 위치를 복원했습니다. 입력 중인 폼 상태와 복귀에 필요한 상태를 구분하고, Redux 기반 상태 저장을 연결했습니다.

조건 변경 후 설정 조회를 갱신하고 기존 목록과 건수 캐시를 제거했습니다. 복귀 시점에 유지할 상태와 조건 변경으로 무효화할 데이터를 나눠 처리했습니다.

목록의 검색 조건과 탐색 위치 저장

→ 상세 또는 작성 화면

→ 목록 복귀 시 상태 복원

조건 변경 → 설정 갱신 → 기존 목록과 건수 캐시 제거

교육 영상(VOD) 서비스

채널, 강좌, 재생, 리뷰와 질문 화면을 구현했습니다. 영상 화면 이탈 전 재생 로그를 저장하고 이전 수강 이력을 이어받도록 했습니다. 질문과 리뷰 작성 뒤 재생 화면으로 돌아오면 해당 탭을 복원하고 관련 조회를 갱신했습니다.

표시할 메뉴는 콘텐츠의 제공 조건과 데이터 유무로 계산했습니다. useMemo와 useEffect를 사용해 표시 조건과 화면 상태를 연결했습니다.

로그인과 앱 내 웹 화면

로그인 후 돌아갈 모바일 URL을 처리하고, WebView 배너를 닫는 여러 경로에서 앱에 종료 상태를 전달했습니다. 중복 노출 기록을 방지하는 참조 상태도 관리했습니다.

검증: Playwright로 실제 라우트와 UI 동작을 확인하고 라우트 맵과 UI 패턴 문서를 작성했습니다. 이 작업의 범위는 브라우저 확인과 문서화입니다.

결과: 화면 이동 전후의 탐색 상태와 교육 영상 수강 흐름을 연결했습니다. App Router 전환, 상태 복원과 영상 재생 화면의 구현을 운영 코드에서 확인할 수 있습니다.

사례 3. 프레임워크와 API 전환 사전 검증

메디게이트 PC와 모바일 / 2026.08 / 상세 API 검증은 다음 쪽

역할: 프레임워크 상향, 공통 API 전환과 레거시 URL 이식 과정의 구현 및 검증을 수행했습니다.

적용 상태: 모바일 소스 편입은 통합 브랜치에 병합했습니다. 아래 프레임워크 상향과 API 전환은 기록 당시 로컬 또는 기능 브랜치 검증 단계입니다.

변경 순서와 기준선

Query v5 전환과 Next.js 15, React 19, Next.js 16 상향을 단계별로 나누고 되돌림 지점을 남겼습니다. Next.js 14의 React 18 의존성 조건 때문에 Next.js 15를 먼저 거쳤으며, 설치 변경과 소스 변경도 나눠 확인했습니다.

기존 타입과 린트 오류를 기준선으로 남겨 새로 생긴 오류와 구분했습니다. 321개 라우트의 비로그인 HTTP 상태 코드 검사에서는 오류 0건을 확인했으며, 브라우저 동작과 서버 로그는 별도로 확인했습니다.

서버의 비동기 API와 동기 코어

Next.js 15의 비동기 쿠키 API 전환 과정에서 쿠키 파싱을 순수 동기 함수로 분리했습니다. 서버 컴포넌트에는 비동기 진입점을, request를 가진 경로에는 동기 진입점을 두어 기존 판별 함수를 불필요하게 비동기로 바꾸지 않도록 했습니다.

서버 컴포넌트: 비동기 쿠키 조회 —
request 보유 경로: 동기 접근 —————> 공통 동기 파싱

HTTP 200 검사에서 보이지 않은 SSR 오류

Next.js 15 상향 중 Suspense fallback이 FormProvider 밖에서 폼 컨텍스트를 읽어 오류가 발생했습니다. 응답은 HTTP 200이어서 상태 코드 검사만으로 드러나지 않았습니다.

Next.js 14와 대조해 재현 조건을 확인했습니다. fallback을 null로 바꿔 폼 컨텍스트 의존을 제거한 뒤 브라우저와 서버 로그를 다시 확인했습니다.

확인 방법	확인한 범위	별도 확인할 부분
기존 오류 기준선 비교	변경으로 생긴 오류 구분	실제 화면 동작
비로그인 HTTP 검사	라우트 응답 상태	로그인과 SSR 회귀
버전 대조와 브라우저 검사	컨텍스트 오류의 재현과 수정	운영 환경 적용

검증: 기능 브랜치에서 버전 대조, 상태 코드 검사와 브라우저 확인을 나눠 수행했습니다. 서버 로그와 콘솔에서 해당 컨텍스트 오류가 사라진 것을 확인했습니다.

결과: 비동기 진입점의 변경 범위를 줄이고, HTTP 성공 응답과 화면 렌더링 성공이 다를 수 있는 회귀를 수정했습니다. 검증 범위는 기능 브랜치입니다.

사례 3 계속. API 계약과 레거시 URL

2026.08 / 기능 브랜치 검증

타입 검사 밖의 API 계약 확인

모바일 API를 PC API로 전환하면서 실제 응답 필드와 화면의 사용 조건을 대조했습니다. 문서와 실제 응답의 차이, 빈 값 전송과 목록 종료 조건은 타입 검사만으로 확인할 수 없었습니다.

확인한 API 계약 결함	접근	수정 범위
댓글 필명 필드 무시	문서와 실제 호출 코드 대조	서버가 받는 필드명으로 수정
빈 객체와 배열 전송 시 실패	서버가 받는 값 확인	미사용 컬렉션의 전송 형식 수정
삭제 글 필터와 목록 종료 판단	원 응답과 필터 결과 비교	필터 전 건수로 종료 판단

브라우저에서 실제 흐름을 확인해 API 계약 결함 3건을 수정했습니다. 서버 응답, 클라이언트 변환과 화면 소비 조건을 구분해서 확인했습니다.

레거시 URL 변환의 순수 함수 이식

기존 URL 변환 규칙을 순수 함수 모듈로 이식했습니다. 고정 매핑과 조건부 규칙을 구분하고, 변환에 사용한 쿼리는 제거하되 나머지 쿼리는 유지했습니다. 목적지에 이미 정해진 값이 있으면 그 값을 우선했습니다.

자기 자신으로 돌아오는 이동은 중단하고 신규 라우트를 가로채는 매핑은 제거했습니다. 변환 함수의 단위 테스트 42건을 기능 브랜치에서 작성했습니다.

기존 경로와 쿼리

- 매핑 또는 조건부 규칙
- 소비한 쿼리 정리, 나머지 쿼리 유지
- 자기 자신으로 돌아오면 이동 중단

전환 경계와 검증의 한계

전환 대상으로 등록한 API 호출만 새 API로 보내고, 대응 API가 없는 기능은 기존 경로를 유지했습니다. 전환 플래그는 빌드 시점에 정해지므로 값 변경 후 재빌드가 필요했습니다.

로컬에서는 결제 요청 페이지와 반환 및 종료 URL의 오리진이 달라 결제창 이후 흐름을 끝까지 확인하지 못했습니다. 라우트 응답 확인과 결제 전 과정 검증을 구분했습니다.

검증: 실제 요청과 응답을 대조한 API 계약 검사, 브라우저 확인, URL 변환 함수의 단위 테스트를 각각 수행했습니다. 단위 테스트 42건은 기능 브랜치에서 Node.js로 직접 실행했습니다.

결과: 정적 검사 밖의 API 계약 결함을 수정하고 URL 변환 규칙을 함수 단위로 확인할 수 있게 했습니다. 프레임워크 상향과 마찬가지로 기록 당시 적용 단계는 기능 브랜치입니다.

백엔드 구현과 팀 개발 기준

메디씨앤씨 / Java와 Spring

2020.02 - 2023.05 / 2022.09부터 FE 개발과 병행

커뮤니티, 채용과 관리 화면의 JSP, API, MyBatis SQL을 직접 개발했습니다. 권한과 중복 조건을 검사하고, 존재하지 않는 게시글과 작성자 불일치 요청을 먼저 차단했습니다.

게시판 토픽의 정렬 순서를 바꿀 때 두 순서 변경 SQL을 서비스 트랜잭션으로 묶었습니다. 채용 검색에서는 지역, 급여, 기관과 진료과 조건을 처리하고 목록과 배너의 공통 조건을 관리했습니다.

채용 공고 스크랩은 요청 중 중복 클릭을 막고 완료 시 상태를 해제했습니다. 화면 입력에서 API와 SQL까지 이어지는 흐름을 직접 다뤘습니다.

팀의 구현 기준과 AI 개발 도구

Figma 디자인 토큰과 스타일 기준을 정리하고 신규 화면의 구현 검토에 활용했습니다. FE 파트리더로 공통 코드의 범위와 앱별 정책을 팀원과 조율하면서 직접 구현을 병행했습니다.

AI 개발 도구에서는 구현과 검토 역할을 나눴습니다. 검토 단계에 계획, 변경 diff와 정적 검사 결과를 제공하도록 하고, 팀에서 사용할 수 있도록 도구를 배포했습니다.

합성 저장소에 넣은 결함 4건을 검출해 검토 흐름을 확인했습니다. 이는 합성 환경의 검증이며 실제 제품의 결함률이나 개발 속도 개선 수치로 해석하지 않습니다.

젠탑 / 웹 서비스와 백엔드 개발

2015.03 - 2019.03

예약 서비스의 화면과 API, 통합환경감시 시스템을 개발했습니다. PHP 기반 기능을 Spring Boot/JSP로 전환하고 Oracle 기록 관리 시스템과 React 화면 연동을 담당했습니다.

화면만 구현하는 범위를 넘어 서버 처리와 데이터 저장까지 연결하는 경험을 쌓았습니다. 이후 의료인 대상 서비스에서도 화면, API와 SQL을 함께 담당했습니다.

경력 전체에서 이어 온 문제 해결 방식

기존 동작과 제약을 확인한 뒤 변경 범위를 나누고, 코드의 책임과 상태의 수명을 구분해 구현했습니다. 화면 응답, 실제 사용자 흐름과 외부 연동은 서로 다른 검증 범위로 관리했습니다.

학력

동양미래대학교 소프트웨어공학과 학사 / 2008.03 - 2015.02